BodyTemp-Blerd

AUTHOR

BDH

Data Set Up

```
# 1.
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.1.4     ✔ readr     2.1.5
✔ forcats   1.0.0     ✔ stringr   1.5.1
✔ ggplot2   4.0.1     ✔ tibble    3.2.1
✔ lubridate 1.9.4     ✔ tidyr     1.3.1
✔ purrr     1.0.4     
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to b
```

```
library(car)
```

```
Loading required package: carData
```

```
Attaching package: 'car'
```

```
The following object is masked from 'package:dplyr':

    recode
```

```
The following object is masked from 'package:purrr':

    some
```

```
library(gt)
library(modelsummary)
library(emmeans)
```

```
Welcome to emmeans.
Caution: You lose important information if you filter this package's results.
See '? untidy'
```

```
library(modelbased)
```

```
Warning: package 'modelbased' was built under R version 4.4.3
```

```
library(multcomp)
```

```
Loading required package: mvtnorm
```

```
Attaching package: 'mvtnorm'
```

```
The following object is masked from 'package:modelbased':

    standardize
```

```
Loading required package: survival
Loading required package: TH.data
Loading required package: MASS
```

```
Attaching package: 'MASS'
```

```
The following object is masked from 'package:dplyr':

    select
```

```
Attaching package: 'TH.data'
```

```
The following object is masked from 'package:MASS':

    geyser
```

```
library(ggpubr)
library(cowplot)
```

```
Attaching package: 'cowplot'
```

```
The following object is masked from 'package:ggpubr':

    get_legend
```

```
The following object is masked from 'package:gt':

    as_gtable
```

```
The following object is masked from 'package:lubridate':

    stamp
```

```
library(lme4)
```

```
Warning: package 'lme4' was built under R version 4.4.3
```

```
Loading required package: Matrix
```

```
Attaching package: 'Matrix'
```

```
The following objects are masked from 'package:tidyr':

    expand, pack, unpack
```

```
library(lmerTest)
```

```
Attaching package: 'lmerTest'
```

```
The following object is masked from 'package:lme4':

    lmer
```

```
The following object is masked from 'package:stats':

    step
```

```
library(outliers)
library(DHARMa)
```

```
This is DHARMa 0.4.7. For overview type '?DHARMa'. For recent changes, type news(package
```

```
library(statmod)
library(readr)
library(dplyr)

# 2.
setwd("/Users/ben/Desktop/Blerd/Data Analysis")

# 3. Load your data
datum = read.csv("Blerd.csv")

# 4. Ensure categorical variables are properly set
datum$SexF = as.factor (datum$Sex)
datum$TreatmentF = as.factor(datum$Treatment)
datum$TimeF = as.factor(datum$Time)
datum$IncubatorF = as.factor (datum$Incubator)
datum$SideF = as.factor (datum$Side)
datum$PostGlucose = as.numeric (datum$PostGlucose)
```

```
Warning: NAs introduced by coercion
```

```
datum$PostKetones = as.numeric (datum$PostKetones)
```

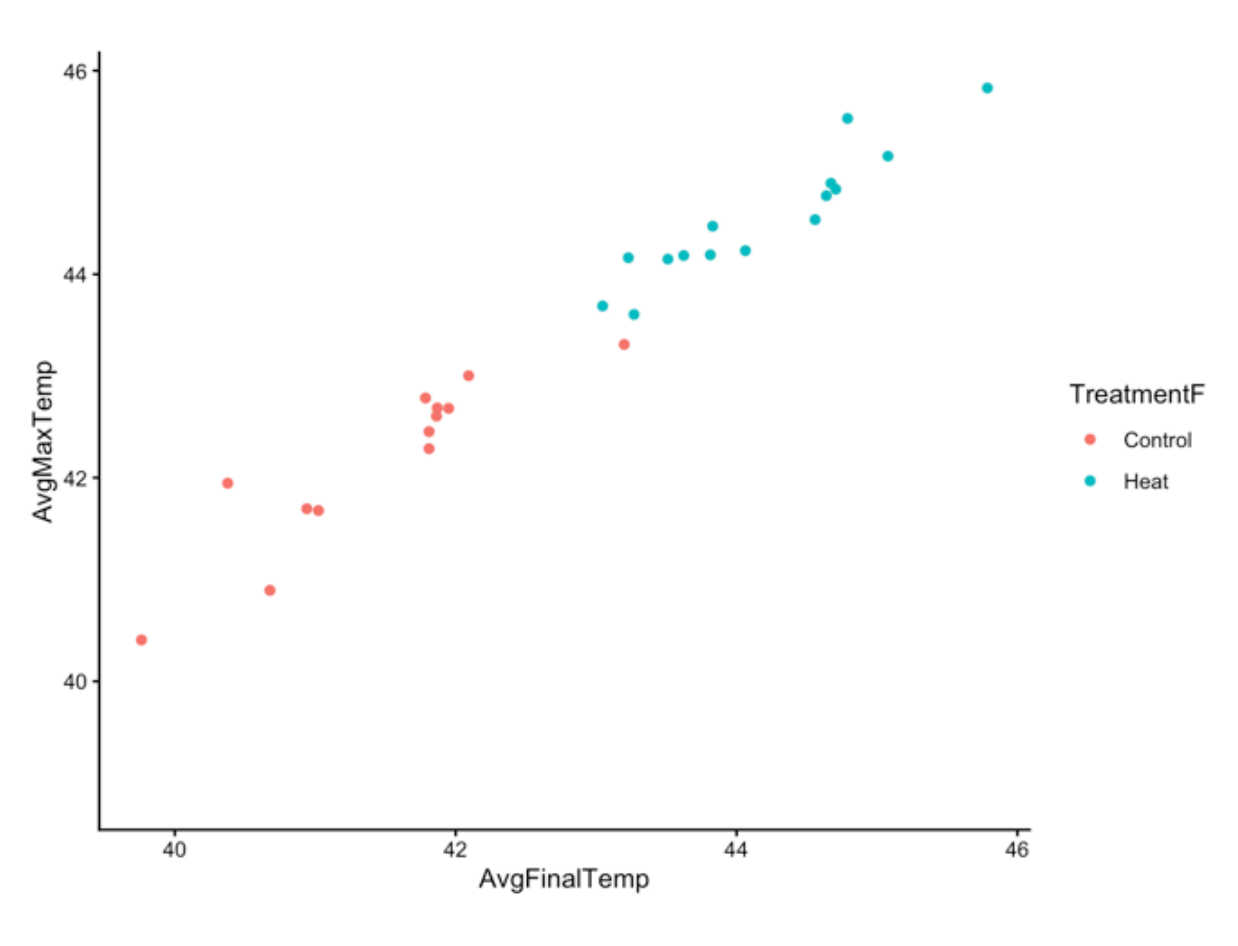
```
Warning: NAs introduced by coercion
```

```
datum$PostDamage = as.numeric (datum$PostDamage)
datum$PostCORT = as.numeric (datum$PostCORT)
datum$AvgFinalTemp = as.numeric (datum$AvgFinalTemp)
datum$AvgMaxTemp = as.numeric (datum$AvgMaxTemp)

# 5. Subset data so there are no dead individuals included
datum1 = datum %>%
  filter(!Died %in% c("Died", "PIT.Death"))

# 6. Take out any of the blank cells that cause problems when graphing
datum2 = datum1 %>%
  filter(PostDamage != "", AvgMaxTemp != "")
```

Initial Data Viz

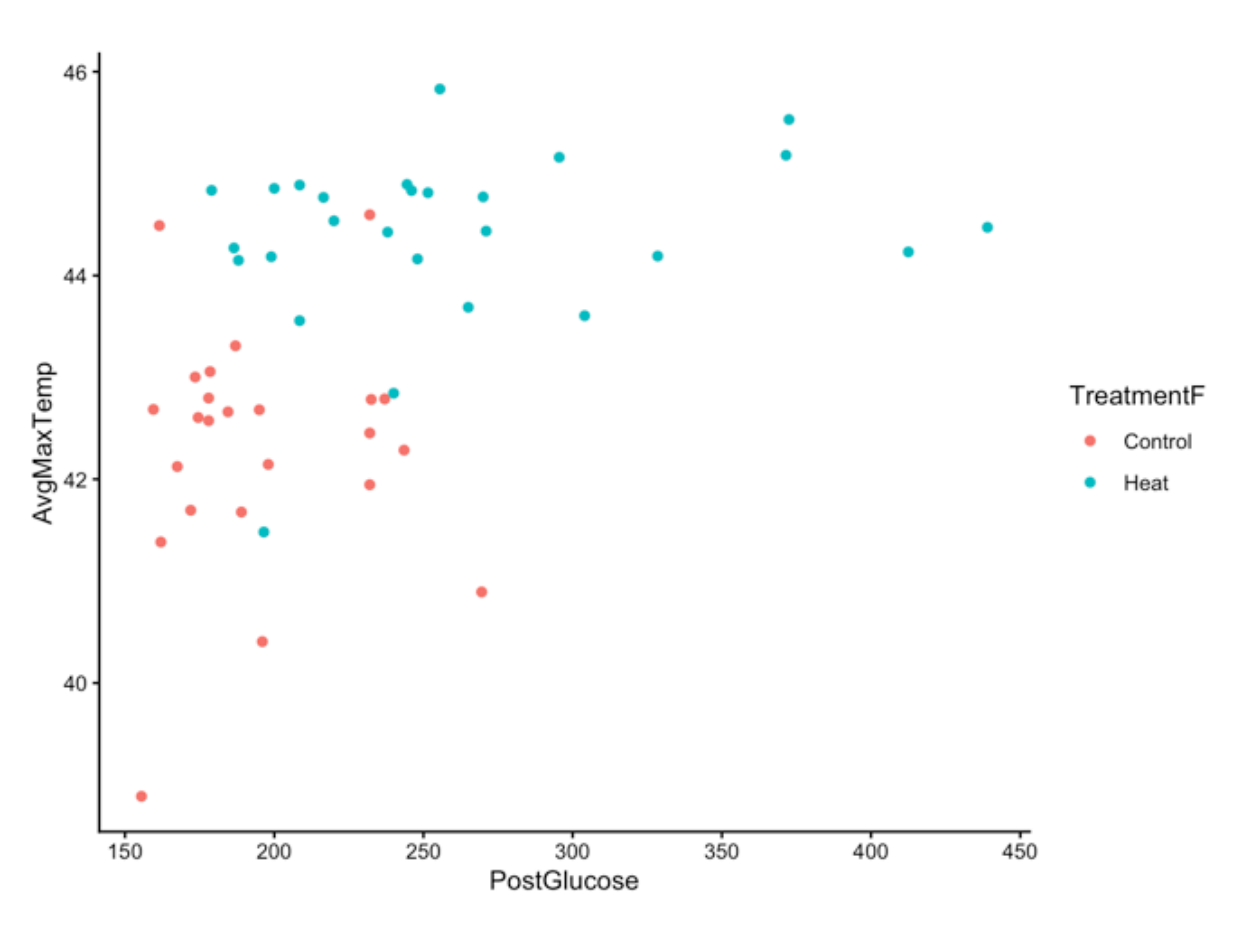
```
# 1. Correlation between the individuals that have both the final 30 min and the top 10%
ggplot(datum2, aes(x = AvgFinalTemp, y = AvgMaxTemp, color = TreatmentF)) +
  geom_point() +
  theme_classic()
```

```
Warning: Removed 25 rows containing missing values or values outside the scale range
(`geom_point()`).
```



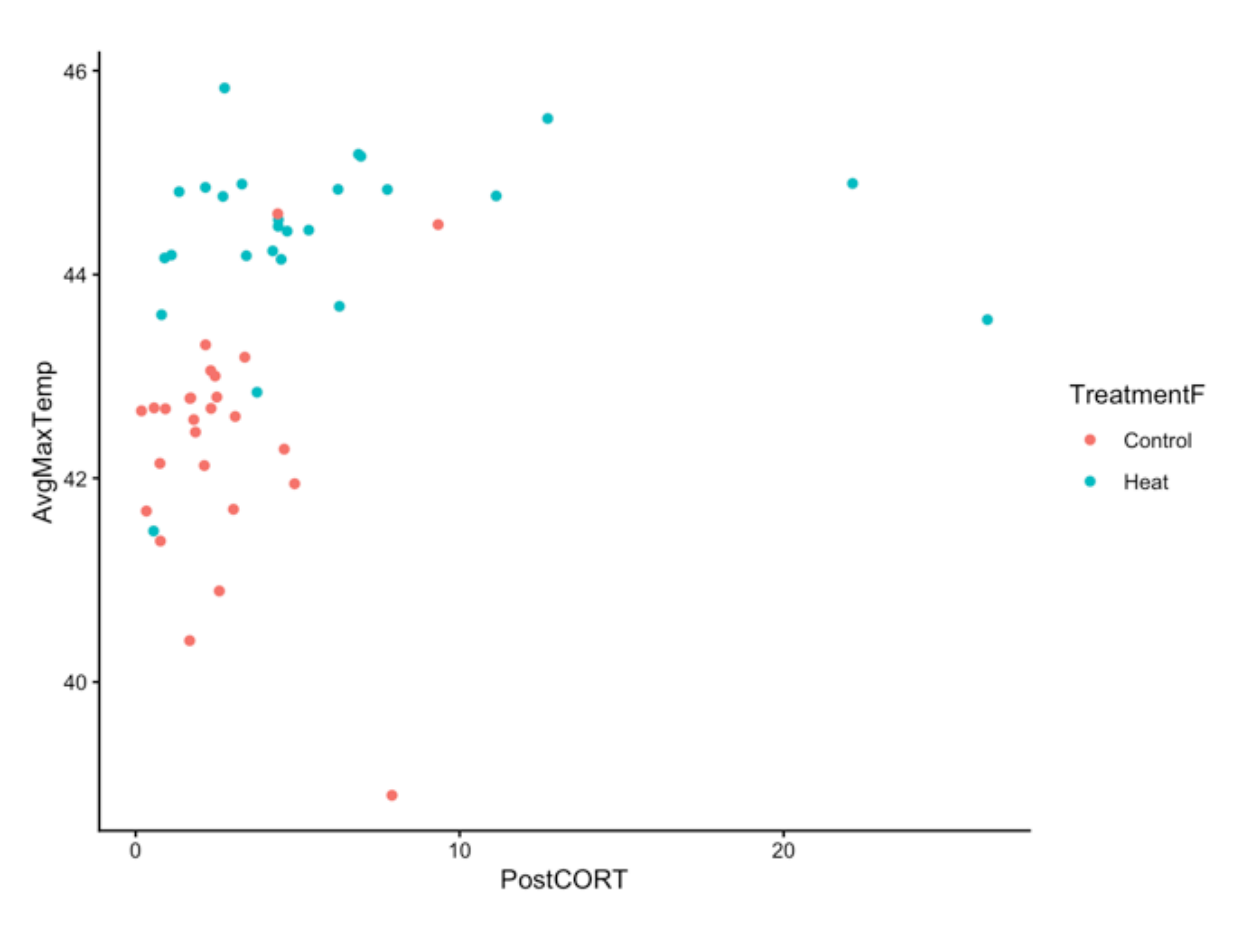
```
# 2. Correlation between the average top 10% of body temperature values and post glucose
ggplot(datum2, aes(x = PostGlucose, y = AvgMaxTemp, color = TreatmentF)) +
  geom_point() +
  theme_classic()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```

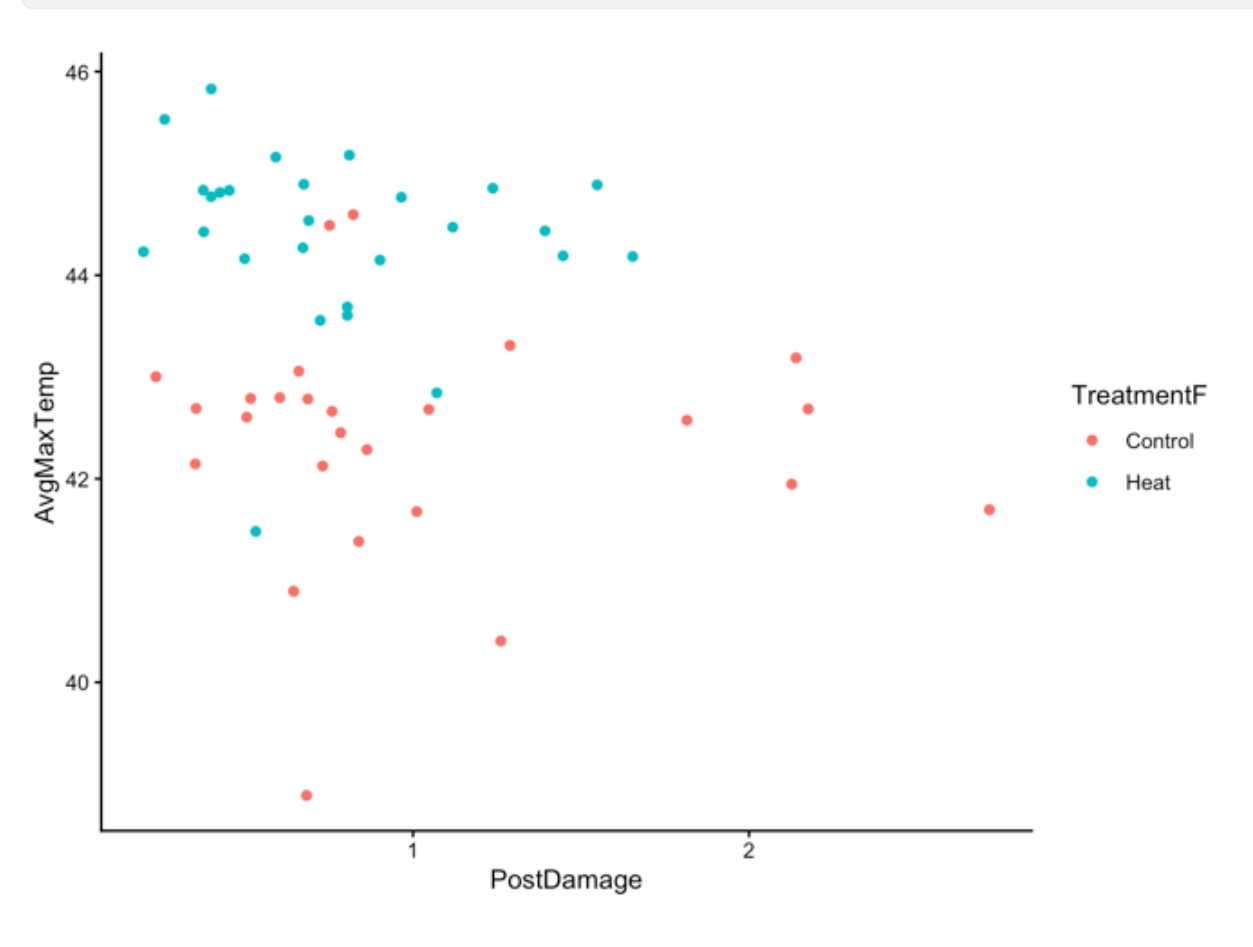


```
# 3. Correlation between the average top 10% of body temperature values and post glucoco
ggplot(datum2, aes(x = PostCORT, y = AvgMaxTemp, color = TreatmentF)) +
  geom_point() +
  theme_classic()
```

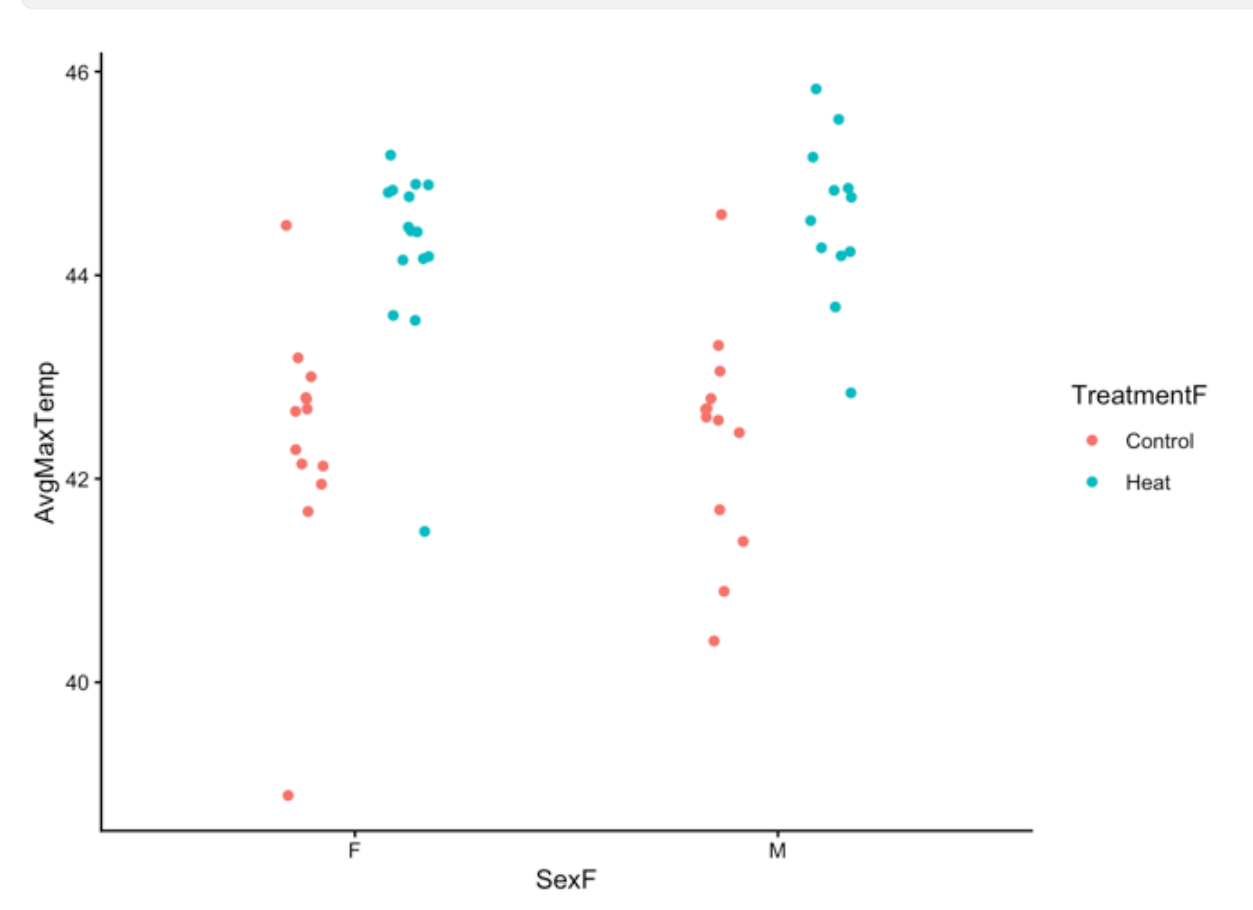
```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_point()`).
```



```
# 4. Correlation between the average top 10% of body temperature values and post DNA dam
ggplot(datum2, aes(x = PostDamage, y = AvgMaxTemp, color = TreatmentF)) +
  geom_point() +
  theme_classic()
```



```
# 5. The average top 10% of body temperature values and relationship to treatment and se
ggplot(datum2, aes(x = SexF, y= AvgMaxTemp, color = TreatmentF)) +
  geom_point(
    position = position_jitterdodge(
      jitter.width = 0.2,
      dodge.width = 0.5
    )
  ) +
  theme_classic()
```



Analysis - Correlation between maximum temperature and final 30 minutes temperature

```
# Run Pearson's Correlation Test
pearson_result = cor.test(
  x = datum2$AvgFinalTemp,    # Your first continuous variable
  y = datum2$AvgMaxTemp,    # Your second continuous variable
  method = "pearson"   # Specify the Pearson method
)

# View the results
print(pearson_result)
```

```

	Pearson's product-moment correlation

data:  datum2$AvgFinalTemp and datum2$AvgMaxTemp
t = 24.623, df = 26, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.9550525 0.9904602
sample estimates:
     cor 
0.979224 
```

Conclusion: Yay – they are correlated! I took this to mean I can use the 'average maximum body temperature variable' to correlate to other variables (Glucose, Damage, ect). This way we can include more body temperature data points.

Analysis - Treatment and sex impact on body temperature

```
#Running the models

#  1. Start with global linear mixed effects model to assess what impacts body temperatu
results = lm(AvgMaxTemp ~ TreatmentF * SexF, data = datum2, na.action = na.omit)
summary(results)
```

```

Call:
lm(formula = AvgMaxTemp ~ TreatmentF * SexF, data = datum2, na.action = na.omit)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4718 -0.3300  0.2047  0.5558  2.2006 

Coefficients:
                     Estimate Std. Error t value Pr(>|t|)    
(Intercept)          42.36001    0.28491 148.678  < 2e-16 ***
TreatmentFHeat        1.89703    0.38926   4.873 1.19e-05 ***
SexFM                 0.03536    0.40293   0.088    0.930    
TreatmentFHeat:SexFM  0.26909    0.56625   0.475    0.637    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.027 on 49 degrees of freedom
Multiple R-squared:  0.5126,	Adjusted R-squared:  0.4828 
F-statistic: 17.18 on 3 and 49 DF,  p-value: 9.313e-08
```

```
Anova(results, type = "III")
```

```
Anova Table (Type III tests)

Response: AvgMaxTemp
                 Sum Sq Df    F value    Pr(>F)    
(Intercept)     23326.8  1 22105.0497 < 2.2e-16 ***
TreatmentF         25.1  1    23.7497 1.193e-05 ***
SexF                0.0  1     0.0077    0.9304    
TreatmentF:SexF     0.2  1     0.2258    0.6367    
Residuals          51.7 49                         
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# 2. The two way interaction was not significant, so it was removed from the model
results2 = lm(AvgMaxTemp ~ TreatmentF + SexF, data = datum2, na.action = na.omit)
summary(results2)
```

```

Call:
lm(formula = AvgMaxTemp ~ TreatmentF + SexF, data = datum2, na.action = na.omit)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4037 -0.2562  0.1426  0.5052  2.1979 

Coefficients:
               Estimate Std. Error t value Pr(>|t|)    
(Intercept)     42.2919     0.2443 173.110  < 2e-16 ***
TreatmentFHeat   2.0242     0.2805   7.216 2.75e-09 ***
SexFM            0.1716     0.2809   0.611    0.544    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.019 on 50 degrees of freedom
Multiple R-squared:  0.5104,	Adjusted R-squared:  0.4908 
F-statistic: 26.06 on 2 and 50 DF,  p-value: 1.764e-08
```

```
Anova(results2, type = "III")
```

```
Anova Table (Type III tests)

Response: AvgMaxTemp
             Sum Sq Df    F value    Pr(>F)    
(Intercept) 31133.8  1 29967.1552 < 2.2e-16 ***
TreatmentF     54.1  1    52.0752 2.754e-09 ***
SexF            0.4  1     0.3732     0.544    
Residuals      51.9 50                         
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#Use AIC to test which one of these models has the best fit
AIC(results, results2)
```

```
         df      AIC
results   5 159.0998
results2  4 157.3435
```

```
#This is the final model, use 'results2' for testing the assumptions.
results_final = lm(AvgMaxTemp ~ TreatmentF + SexF, data = datum2, na.action = na.omit)
summary(results_final)
```

```

Call:
lm(formula = AvgMaxTemp ~ TreatmentF + SexF, data = datum2, na.action = na.omit)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4037 -0.2562  0.1426  0.5052  2.1979 

Coefficients:
               Estimate Std. Error t value Pr(>|t|)    
(Intercept)     42.2919     0.2443 173.110  < 2e-16 ***
TreatmentFHeat   2.0242     0.2805   7.216 2.75e-09 ***
SexFM            0.1716     0.2809   0.611    0.544    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.019 on 50 degrees of freedom
Multiple R-squared:  0.5104,	Adjusted R-squared:  0.4908 
F-statistic: 26.06 on 2 and 50 DF,  p-value: 1.764e-08
```

```
Anova(results_final, type = "III")
```

```
Anova Table (Type III tests)

Response: AvgMaxTemp
             Sum Sq Df    F value    Pr(>F)    
(Intercept) 31133.8  1 29967.1552 < 2.2e-16 ***
TreatmentF     54.1  1    52.0752 2.754e-09 ***
SexF            0.4  1     0.3732     0.544    
Residuals      51.9 50                         
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#Assumptions of the model

# 1. Check assumptions of a linear model

# --- Extract Residuals and Fitted Values ---
#1. Extract the data frame used by the model
datum_clean <- model.frame(results_final)

# 2. Calculate and assign the fitted and residual values to this clean data frame
datum_clean$fitted <- fitted(results_final)
datum_clean$raw_resid <- residuals(results_final)
datum_clean$std_resid <- datum_clean$raw_resid / sd(datum_clean$raw_resid, na.rm = TRUE)

# 3. Use the studentized residuals method from the 'lme4' utility functions
# Although 'rstandard' doesn't work, 'residuals' is robust.
# If you really need a standardized residual, use this function:
datum_clean$std_resid <- datum_clean$raw_resid / sd(datum_clean$raw_resid, na.rm = TRUE)


# --- 2. Create the Plots ---

# A. Linearity and Homoscedasticity Plot (Residuals vs. Fitted)
ggplot(datum_clean, aes(x = fitted, y = std_resid)) +
  geom_point(alpha = 0.6) +
  # Add the ideal reference line at y=0
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  # Add a LOESS smoothing line (red line in performance)
  geom_smooth(method = "loess", se = FALSE, color = "blue", linewidth = 0.5) +
  labs(
    title = "Linearity Check: Standardized Residuals vs. Fitted Values",
    x = "Fitted Values",
    y = "Standardized Residuals"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(size = 10))
```

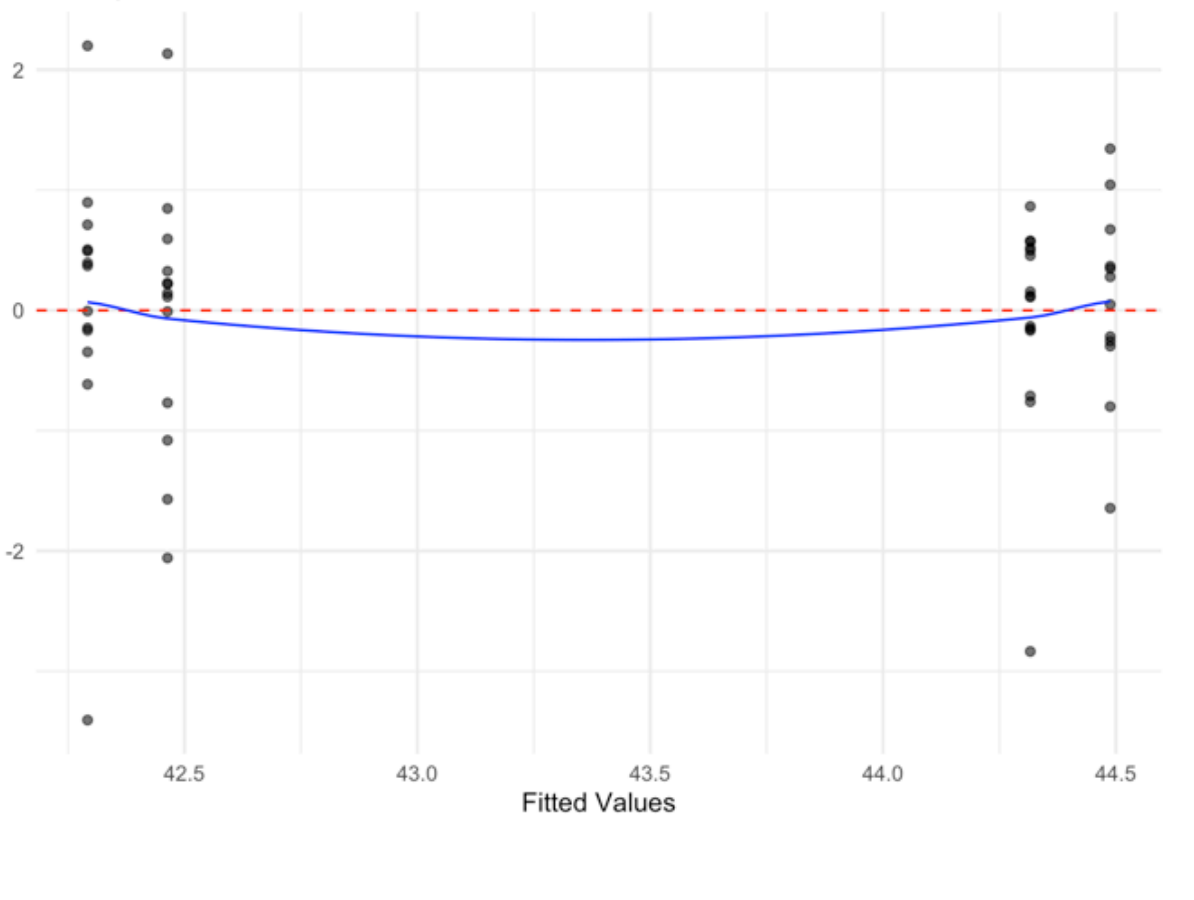
```
`geom_smooth()` using formula = 'y ~ x'
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 42.281
```

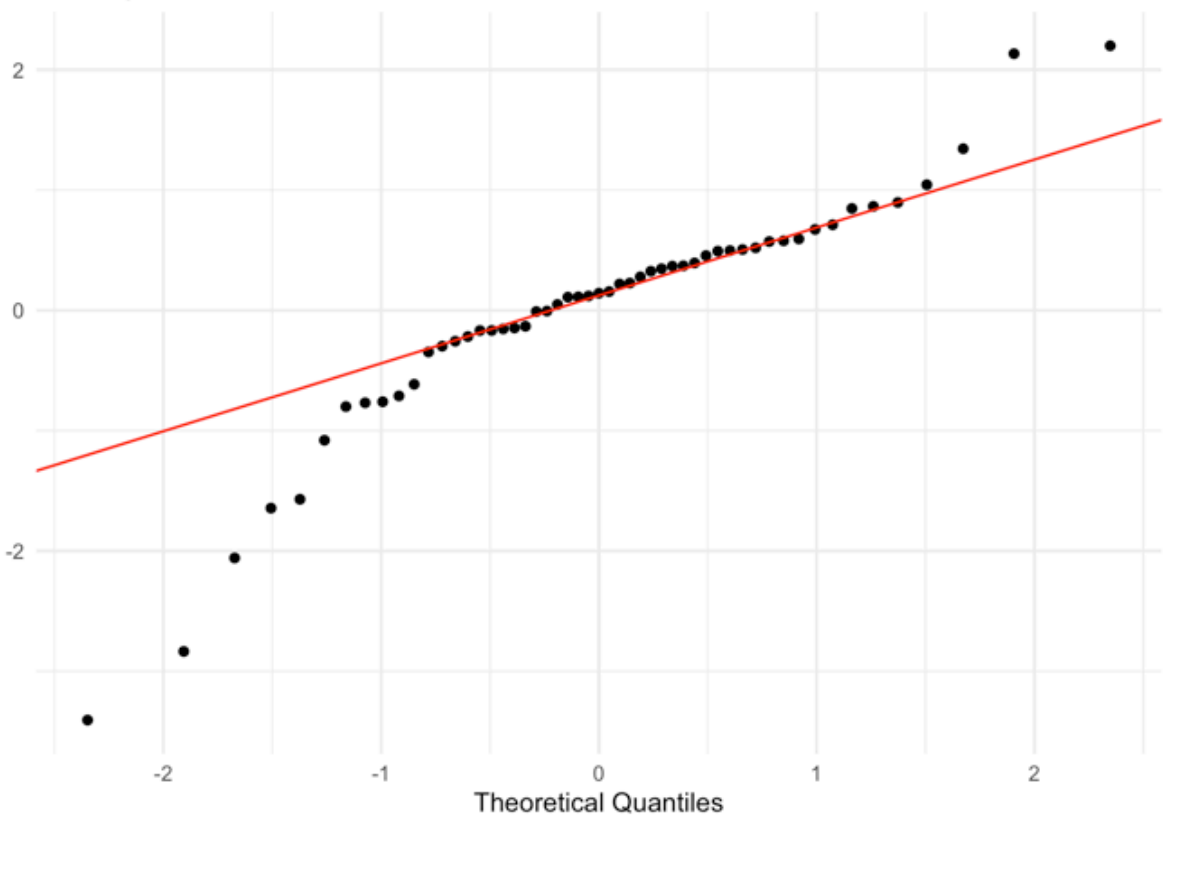
```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 2.0352
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 5.8299e-15
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 4.1419
```



```
# B. Normality Check (Quantile-Quantile Plot)
ggplot(datum_clean, aes(sample = std_resid)) +
  # Create the Q-Q plot
  stat_qq() +
  # Add the reference line (the ideal normal distribution line)
  stat_qq_line(color = "red") +
  labs(
    title = "Normality Check: Q-Q Plot of Standardized Residuals",
    x = "Theoretical Quantiles",
    y = "Standardized Residuals"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(size = 10))
```



```
# C. Scale-Location Plot (Homogeneity of Variance Check)
# This uses the square root of the absolute standardized residuals
ggplot(datum_clean, aes(x = fitted, y = sqrt(abs(std_resid)))) +
  geom_point(alpha = 0.6) +
  # Add the ideal flat reference line
  geom_smooth(method = "loess", se = FALSE, color = "blue", linewidth = 0.5) +
  labs(
    title = "Homogeneity of Variance Check: Scale-Location Plot",
    x = "Fitted Values",
    y = expression(sqrt(abs("Standardized Residuals")))
  ) +
  theme_minimal() +
  theme(plot.title = element_text(size = 10))
```

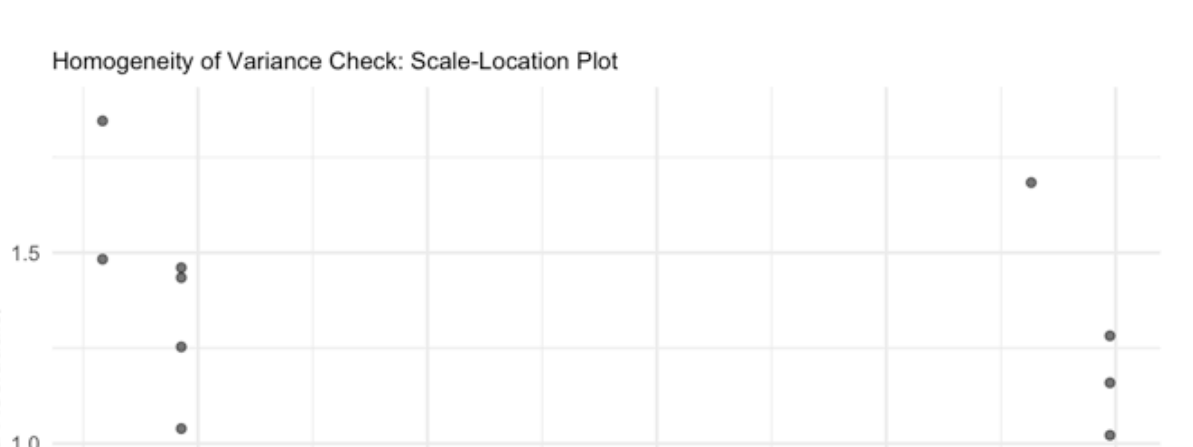
```
`geom_smooth()` using formula = 'y ~ x'
```

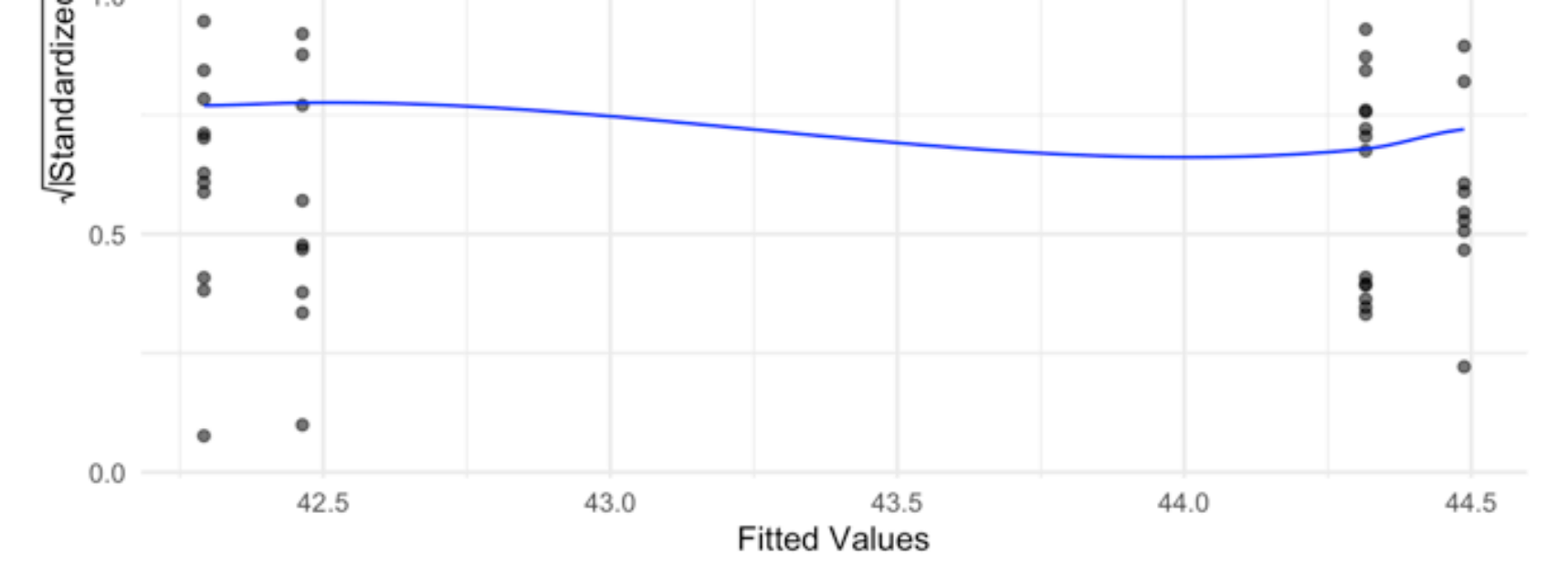
```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 42.281
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 2.0352
```

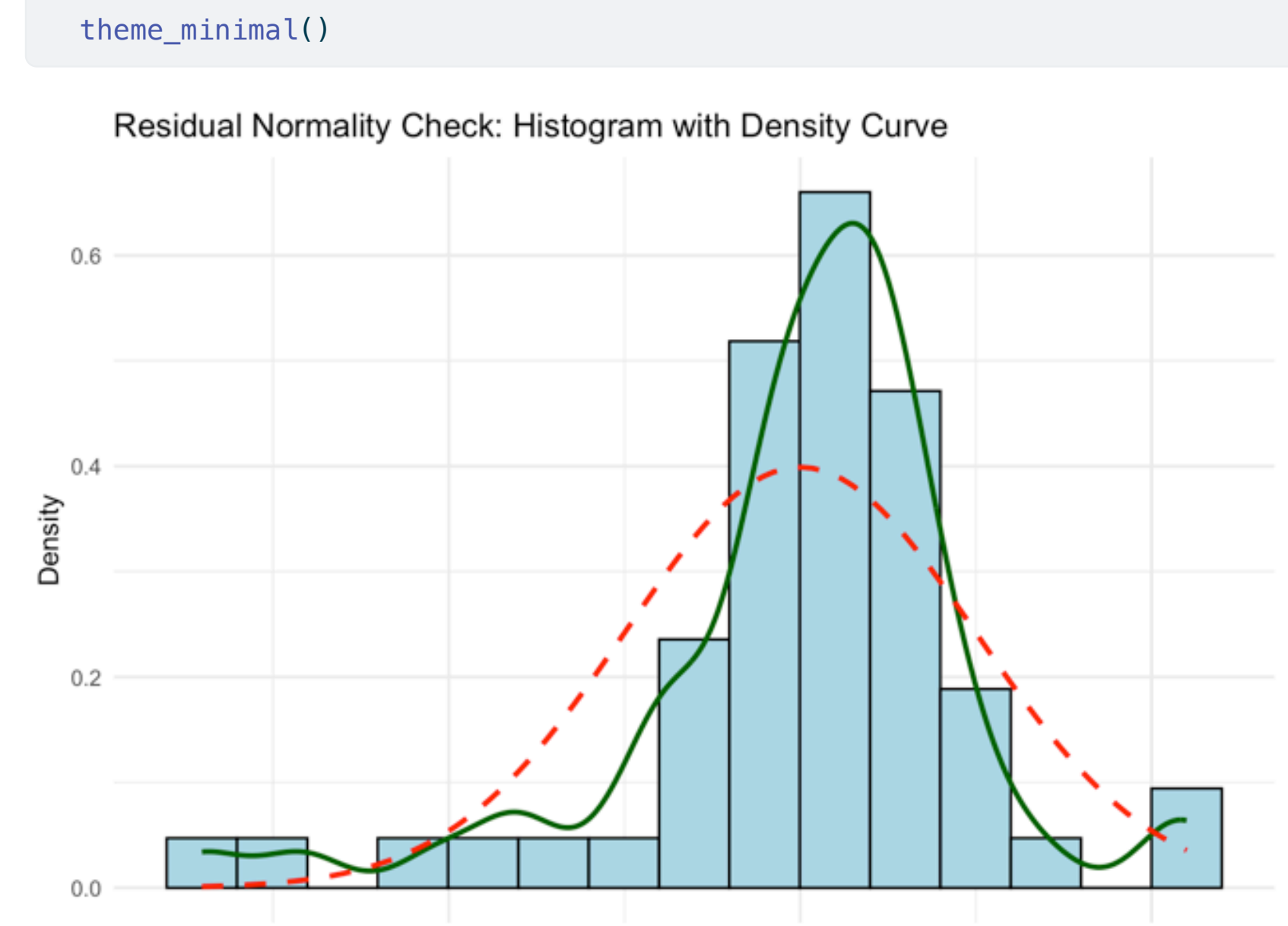
```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 5.8299e-15
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 4.1419
```





```r
#D. Histogram of residuals
# Calculate mean and standard deviation of residuals for the density curve
resid_mean <- mean(datum_clean$std_resid, na.rm = TRUE)
resid_sd <- sd(datum_clean$std_resid, na.rm = TRUE)

ggplot(datum_clean, aes(x = std_resid)) +
  # 1. Create the Histogram (using density scale)
  geom_histogram(aes(y = after_stat(density)),
               bins = 15, # Adjust bin count for best visualization
               color = "black",
               fill = "lightblue") +

  # 2. Overlay the Density of the Residuals (Green Line)
  geom_density(color = "darkgreen", linewidth = 1) +

  # 3. Overlay the Theoretical Normal Density Curve (Red Dashed Line)
  stat_function(fun = dnorm,
              args = list(mean = resid_mean, sd = resid_sd),
              color = "red",
              linewidth = 1,
              linetype = "dashed") +

  labs(
    title = "Residual Normality Check: Histogram with Density Curve",
    x = "Standardized Residuals (Z-Scores)",
    y = "Density"
  ) +
  theme_minimal()
```



Conclusions: Based on the residuals from the model, I think the assumptions are met and we can continue with analysis using this model.

Next steps: No further steps

Manuscript Graph

```r
# 1. Add the means and 95% CIs to the dataframe for the variables we are interested in

datum2_with_summary = datum2 %>%

  # 1. Group the data by the interaction terms
  group_by(TreatmentF) %>%

  # 2. Calculate the group-level statistics (Mean, CI, etc.)
  mutate(
    N_group = n(),
    Mean_Temp = mean(AvgMaxTemp, na.rm = TRUE),
    SD_Temp = sd(AvgMaxTemp, na.rm = TRUE),
    SE_Temp = SD_Temp / sqrt(N_group),
    CI_Temp = 1.96 * SE_Temp,
    CI_Upper_Group = Mean_Temp + CI_Temp,
    CI_Lower_Group = Mean_Temp - CI_Temp
  ) %>%
  ungroup()
  #View the dataframe that was created
head(datum2_with_summary)
```

```
# A tibble: 6 × 40
     ID Sex   Treatment Incubator Side  Time  Died  ReinsertPIT IntakeMass
  <int> <chr> <chr>     <chr>     <chr> <chr> <chr> <chr>            <dbl>
1  4820 M     Control   Zeus      Right Early No    ""                14.3
2  4823 M     Heat      Aphrodite Right Late  No    ""                12.3
3  4824 M     Heat      Apollo    Left  Early No    "Yes"             12.5
4  4826 M     Control   Apollo    Left  Early No    ""                14.2
5  4829 M     Heat      Artemis   Left  Late  No    ""                14.3
6  4830 M     Heat      Zeus      Right Early No    ""                16.5
# ℹ 31 more variables: IntakeTarsus <dbl>, PreDate <chr>, PITtagID <chr>,
#   PreMass <dbl>, PreGlucose <dbl>, PreKetones <dbl>, PreHCT <dbl>,
#   PreCORT <dbl>, PreDamage <dbl>, AccDate <chr>, AccMass <dbl>,
#   PostDate <chr>, PostGlucose <dbl>, PostKetones <dbl>, PostHemo <chr>,
#   PostCORT <dbl>, PostDamage <dbl>, AvgFinalTemp <dbl>, AvgMaxTemp <dbl>,
#   SexF <fct>, TreatmentF <fct>, TimeF <fct>, IncubatorF <fct>, SideF <fct>,
#   N_group <int>, Mean_Temp <dbl>, SD_Temp <dbl>, SE_Temp <dbl>, …
```

```r
# 2. Cleaning up data frame before plotting
  # Define the consistent dodge width and rename the data frame for plotting
plot_data = datum2_with_summary

  #Order the time points so that Pre and Post are graphed in the correct order on the pl
plot_data$TreatmentF = as.factor(plot_data$TreatmentF)

# 3. Plotting the graph

plot = ggplot(data = plot_data, aes(x = TreatmentF, y = AvgMaxTemp, color = TreatmentF))
  geom_jitter(
    size = 3.5,
    alpha = 0.5,
    position = position_jitter(width = 0.2)
  ) +

  geom_point(
    aes(y = Mean_Temp, group = TreatmentF),
    size = 7.5,
    stroke = 1,
    color = "black"
  ) +

  geom_errorbar(
    aes(ymin = CI_Lower_Group, ymax = CI_Upper_Group, group = TreatmentF),
    width = 0.2,
    linewidth = 1,
    color = "black",
  ) +

  geom_line(
    aes(y = Mean_Temp, group = TreatmentF),
    linewidth = 1.5,
  ) +

  scale_color_manual(
    values = c(
      "Control" = "#AAB8C2",
      "Heat" = "firebrick4"
    )
  ) +

  labs(
    x = "Treatment",
    y = "Body Temperature (°C)",
    color = "Treatment Group"
  ) +
  theme_classic() +

theme(
  axis.title = element_text(face = "bold", size = 30),
  axis.text = element_text(size = 25),
  strip.text = element_text(size = 20),
  panel.background = element_rect(fill = "transparent", color = NA),
  plot.background = element_rect(fill = "transparent", color = NA),
  axis.line = element_line(linewidth = 1.5, colour = "black"),
  legend.position = "none"
)


# 4. Adding signifigance and sample size

plot1 =  plot +
  annotate(
    "text",
    x = c(1, 2),
    y = c(46.6, 47.8),
    label = c("a", "b"),
    size = 12,
    color = "black") +

   # Add a new layer for the sample sizes
  annotate(
    "text",
    x = c(1, 2),
    y = c(45.6, 46.8),
    # Add your sample sizes here
    label = c("(26)", "(27)"),
    size = 10, # Make the font size slightly smaller
    color = "black")

# 5. Save the graph
ggsave(
  filename = "TempPlot.png",
  plot = plot1,
  width = 6.49,
  height = 6.49,
  units = "in",
  dpi = 500,
  bg = "transparent"
)
```

Manuscript Graph - With sex

```r
#Manuscript Graph with Sex

# 1. Add the means and 95% CIs to the dataframe for the variables we are interested in

datum2_with_summary = datum2 %>%

  # 1. Group the data by the interaction terms
  group_by(TreatmentF) %>%

  # 2. Calculate the group-level statistics (Mean, CI, etc.)
  mutate(
    N_group = n(),
    Mean_Temp = mean(AvgMaxTemp, na.rm = TRUE),
    SD_Temp = sd(AvgMaxTemp, na.rm = TRUE),
    SE_Temp = SD_Temp / sqrt(N_group),
    CI_Temp = 1.96 * SE_Temp,
    CI_Upper_Group = Mean_Temp + CI_Temp,
    CI_Lower_Group = Mean_Temp - CI_Temp
  ) %>%
  ungroup()
  #View the dataframe that was created
head(datum2_with_summary)
```

```
# A tibble: 6 × 40
     ID Sex   Treatment Incubator Side  Time  Died  ReinsertPIT IntakeMass
  <int> <chr> <chr>     <chr>     <chr> <chr> <chr> <chr>            <dbl>
1  4820 M     Control   Zeus      Right Early No    ""                14.3
2  4823 M     Heat      Aphrodite Right Late  No    ""                12.3
3  4824 M     Heat      Apollo    Left  Early No    "Yes"             12.5
4  4826 M     Control   Apollo    Left  Early No    ""                14.2
5  4829 M     Heat      Artemis   Left  Late  No    ""                14.3
6  4830 M     Heat      Zeus      Right Early No    ""                16.5
# ℹ 31 more variables: IntakeTarsus <dbl>, PreDate <chr>, PITtagID <chr>,
#   PreMass <dbl>, PreGlucose <dbl>, PreKetones <dbl>, PreHCT <dbl>,
#   PreCORT <dbl>, PreDamage <dbl>, AccDate <chr>, AccMass <dbl>,
#   PostDate <chr>, PostGlucose <dbl>, PostKetones <dbl>, PostHemo <chr>,
#   PostCORT <dbl>, PostDamage <dbl>, AvgFinalTemp <dbl>, AvgMaxTemp <dbl>,
#   SexF <fct>, TreatmentF <fct>, TimeF <fct>, IncubatorF <fct>, SideF <fct>,
#   N_group <int>, Mean_Temp <dbl>, SD_Temp <dbl>, SE_Temp <dbl>, …
```

```r
# 2. Cleaning up data frame before plotting
  # Define the consistent dodge width and rename the data frame for plotting
plot_data = datum2_with_summary

  #Order the time points so that Pre and Post are graphed in the correct order on the pl
plot_data$TreatmentF = as.factor(plot_data$TreatmentF)

# 3. Plotting the graph

plot = ggplot(data = plot_data, aes(x = TreatmentF, y = AvgMaxTemp, color = TreatmentF))
  geom_jitter(
    aes(shape = SexF),
    size = 3.5,
    alpha = 0.5,
    position = position_jitter(width = 0.2)
  ) +

  geom_point(
    aes(y = Mean_Temp, group = TreatmentF),
    shape = 18,
    size = 7.5,
    stroke = 1,
    color = "black"
  ) +

  geom_errorbar(
    aes(ymin = CI_Lower_Group, ymax = CI_Upper_Group, group = TreatmentF),
    width = 0.2,
    linewidth = 1,
    color = "black",
  ) +

  geom_line(
    aes(y = Mean_Temp, group = TreatmentF),
    linewidth = 1.5,
  ) +

  scale_color_manual(
    values = c(
      "Control" = "#AAB8C2",
      "Heat" = "firebrick4"
    )
  ) +

  labs(
    x = "Treatment",
    y = "Body Temperature (°C)",
    color = "Treatment Group"
  ) +
  theme_classic() +

theme(
  axis.title = element_text(face = "bold", size = 30),
  axis.text = element_text(size = 25),
  strip.text = element_text(size = 20),
  panel.background = element_rect(fill = "transparent", color = NA),
  plot.background = element_rect(fill = "transparent", color = NA),
  axis.line = element_line(linewidth = 1.5, colour = "black"),
  legend.position = "none"
)


# 4. Adding signifigance and sample size

plot1 =  plot +
  annotate(
    "text",
    x = c(1, 2),
    y = c(46.6, 47.8),
    label = c("a", "b"),
    size = 12,
    color = "black") +

   # Add a new layer for the sample sizes
  annotate(
    "text",
    x = c(1, 2),
    y = c(45.6, 46.8),
    # Add your sample sizes here
    label = c("(26)", "(27)"),
    size = 10, # Make the font size slightly smaller
    color = "black")

# 5. Save the graph
ggsave(
  filename = "TempPlotSex.png",
  plot = plot1,
  width = 6.49,
  height = 6.49,
  units = "in",
  dpi = 500,
  bg = "transparent"
)
```